

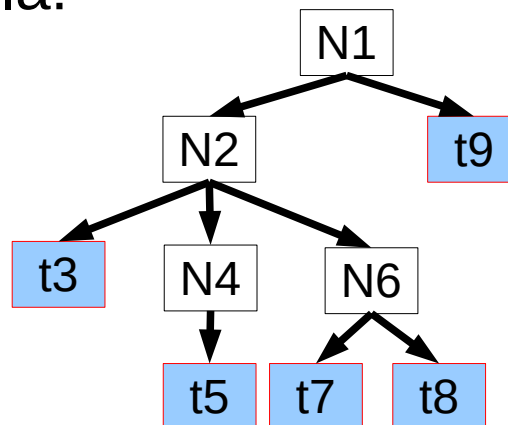
**RI402**  
**Dizajn kompajlera**  
dr.sc. Edin Pjanić

# Pregled predavanja

- Metode parsiranja
  - Top-down – metoda rekurzivnog spuštanja  
(*recursive descent parsing*)
- Lijeva rekurzija i njena eliminacija
- Lijevo faktorisanje

# Metoda rekurzivnog spuštanja (Recursive Descent Parsing - RDP)

- Ovo je top-down metoda (s vrha nadole).
- Stablo parsiranja se konstruiše:
  - S vrha
  - S lijeva nadesno
- Koristi se backtracking.
- Provjeravaju se redom sve produkcije dok se ne pronade kombinacija koja prihvata string. U suprotnom se string odbija.
- Primjer stabla parsiranja za neku ulaznu sekvencu neterminalnih (N) i terminalnih (t) simbola:



Ako je ulazna sekvencu tokena:  
[t3, t5, t7, t8, t9]  
parser će je prihvatiti, s obzirom  
na dobijeno stablo parsiranja.

# Primjer – demonstracija RDP

- Za gramatiku:

$$E \rightarrow T \mid T + E$$
$$T \rightarrow \text{num} * T \mid \text{num} \mid ( E )$$

- Za ulaz: "( 5 )", imaćemo ulaznu sekvencu tokena:

( num<sub>5</sub> )

Imaćemo:

E

→ T

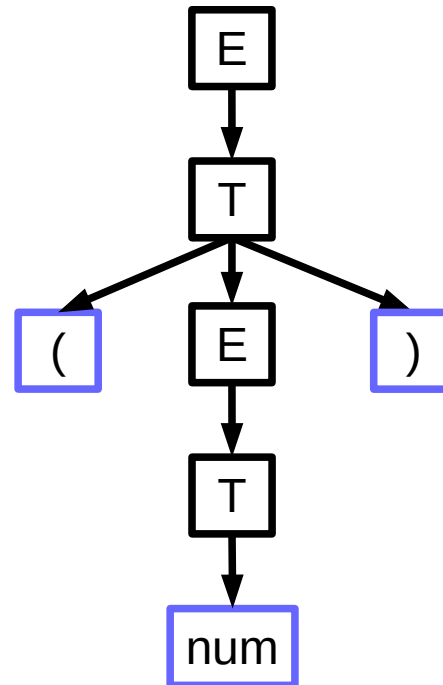
→ num \* T

→ num

→ ( E )

→ ( T )

→ ( num )



Uvijek provjeravamo (koristimo) produkcijska pravila neterminala u redoslijedu kojim su napisana.

Svaki put kad dobijemo terminal na mjestu lista, provjerimo poklapanje sa ulaznom sekvencom tokena. U slučaju nepoklapanja radimo **backtracking**.

# RDP - Algoritam



Definiše se serija bool funkcija koje provjeravaju poklapanje ulazne sekvence tokena sa:

- Proizvedenim terminalom (kod isti kao klasa tokena)

```
bool terminal(TOKEN term) { return term == tokens[next++] ; }
```

- i-tom produkcijom svakog neterminala S

```
bool Si() { ... }
```

- Provjera za sve produkcije od S:

```
bool S() {  
    ...  
    return    ... S1()  
              || ... S2()  
              || ...  
}
```

|         |
|---------|
| S -> S1 |
| S2      |
| ...     |
| Sn      |

< objašnjenje na tabli i primjeru s

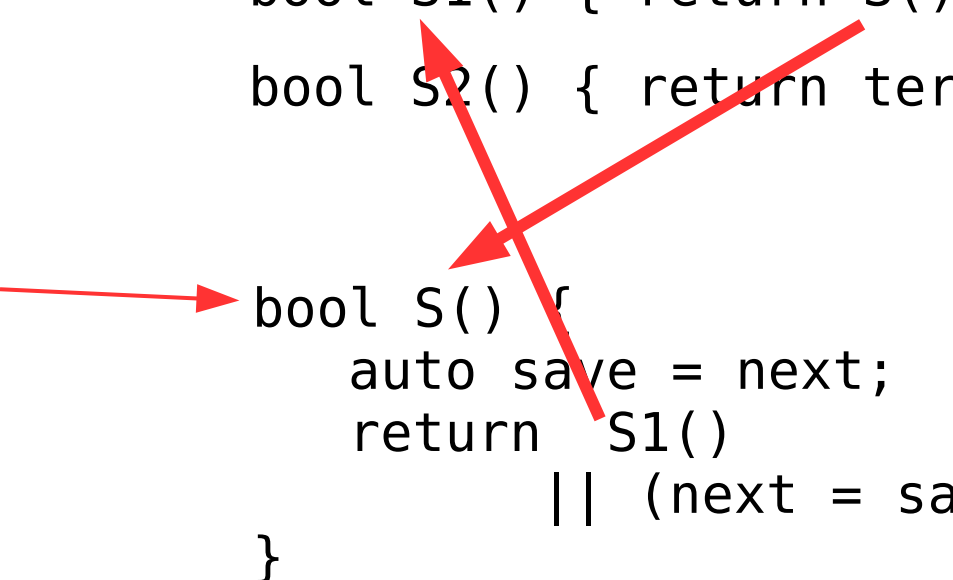
# Problem sa RDP

Posmatrajmo produkcijska pravila:

$S \rightarrow S a \mid b$

- Za funkcije koje provjeravaju poklapanje ulaza sa produkcijama bismo imali:

```
bool S1() { return S() && terminal( a ) ; }  
bool S2() { return terminal( b ) ; }  
  
bool S() {  
    auto save = next;  
    return S1()  
        || (next = save, S2());  
}
```



Imamo beskonačnu rekurziju.

# Lijeva rekurzija

- Gramatika je *lijevo rekurzivna* ako za neko  $\alpha$  postoji derivacija

$$\blacksquare A \Rightarrow^+ A \alpha$$

- Primjeri rekurzivnih gramatika:

$$S \rightarrow S a \mid b$$

$$S \rightarrow T a \mid b$$

$$T \rightarrow U c \mid d$$

$$U \rightarrow S e \mid f$$

$$S \rightarrow T U \mid a$$

$$T \rightarrow b \mid \varepsilon$$

$$U \rightarrow S c$$

# Eliminacija lijeve rekurzije

- Par produkcijskih pravila
  - $A \rightarrow A\alpha \mid \beta$

može se zamijeniti sa:

- $A \rightarrow \beta A'$
- $A' \rightarrow \alpha A' \mid \varepsilon$



# Eliminacija lijeve rekurzije

- U opštijem slučaju, ako imamo produkcije:

$$\square A \rightarrow A \alpha_1 \mid \dots \mid A \alpha_n \mid \beta_1 \mid \dots \mid \beta_m$$

možemo ih zamijeniti sa:

$$\square A \rightarrow \beta_1 A' \mid \dots \mid \beta_m A'$$

$$\square A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_n A' \mid \varepsilon$$

- Ovo dovodi do promjene stabla parsiranja, ali više nema lijeve rekurzije.

# Eliminacija lijeve rekurzije

- Primjer:
  - $E \rightarrow E + T \mid T$
  - $T \rightarrow T * F \mid F$
  - $F \rightarrow ( E ) \mid \text{num}$

Eliminacijom lijeve rekurzije imaćemo:

- $E \rightarrow T E'$
- $E' \rightarrow + T E' \mid \varepsilon$
- $T \rightarrow F T'$
- $T' \rightarrow * F T' \mid \varepsilon$
- $F \rightarrow ( E ) \mid \text{num}$

# Lijevo faktorisanje

- Posmatrajmo naredna produkcijska pravila:
  - $T \rightarrow \text{int} * T \mid \text{int} \mid ( E )$
- U slučaju terminalnog simbola **int** na ulazu u parser, koje produkcijsko pravilo ćemo koristiti?
- Ideja lijevog faktorisanja:
  - eliminisati zajedničke prefikse
  - odluku koje produkcijsko pravilo koristiti odgađamo za kasnije, kad dobijemo više informacija.

# Lijevo faktorisiranje

- U opštijem slučaju, ako imamo produkcije:

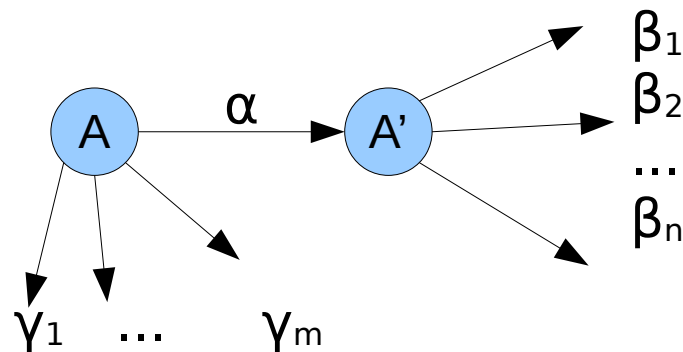
$$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \dots \mid \alpha \beta_n \mid \gamma_1 \mid \dots \mid \gamma_m$$



možemo ih zamijeniti sa:

$$A \rightarrow \alpha A' \mid \gamma_1 \mid \dots \mid \gamma_m$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$



# Eliminacija lijeve rekurzije i lijevo faktorisanje

- Primjer gramatike:

- $S \rightarrow E ; S \mid E ;$
- $E \rightarrow E + T \mid T$
- $T \rightarrow \text{num} * T \mid ( E ) \mid \text{num}$

Eliminacijom lijeve rekurzije i nakon lijevog faktorisanja imaćemo gramatiku:

- $S \rightarrow E ; S'$
- $S' \rightarrow S \mid \epsilon$
- $E \rightarrow T E'$
- $E' \rightarrow + T E' \mid \epsilon$
- $T \rightarrow \text{num} T' \mid ( E )$
- $T' \rightarrow * T \mid \epsilon$

Prilikom implementacije funkcija za neterminalne simbole voditi računa o redoslijedu provjere produkcijskih pravila:

*ako neko produkcijsko pravilo kao prefiks sadrži neko drugo pravilo, prvo provjeriti mogućnost upotrebe dužeg pa onda kraćeg pravila.*

**Za vježbu:** Napisati program koji sa standardnog ulaza uzima izraze i provjerava ispravnost u skladu sa datom gramatikom (aritmetički izraz).

Parser implementirati kao RDP.